

ECE/CS 757 Advanced Computer Architecture II

Spring 2025

Consistency (Part 3)

Instructor: Prof. Joshua San Miguel

Some slides adopted from Greg Steffan, Natalie Enright Jerger and Mikko Lipasti

Last Lecture

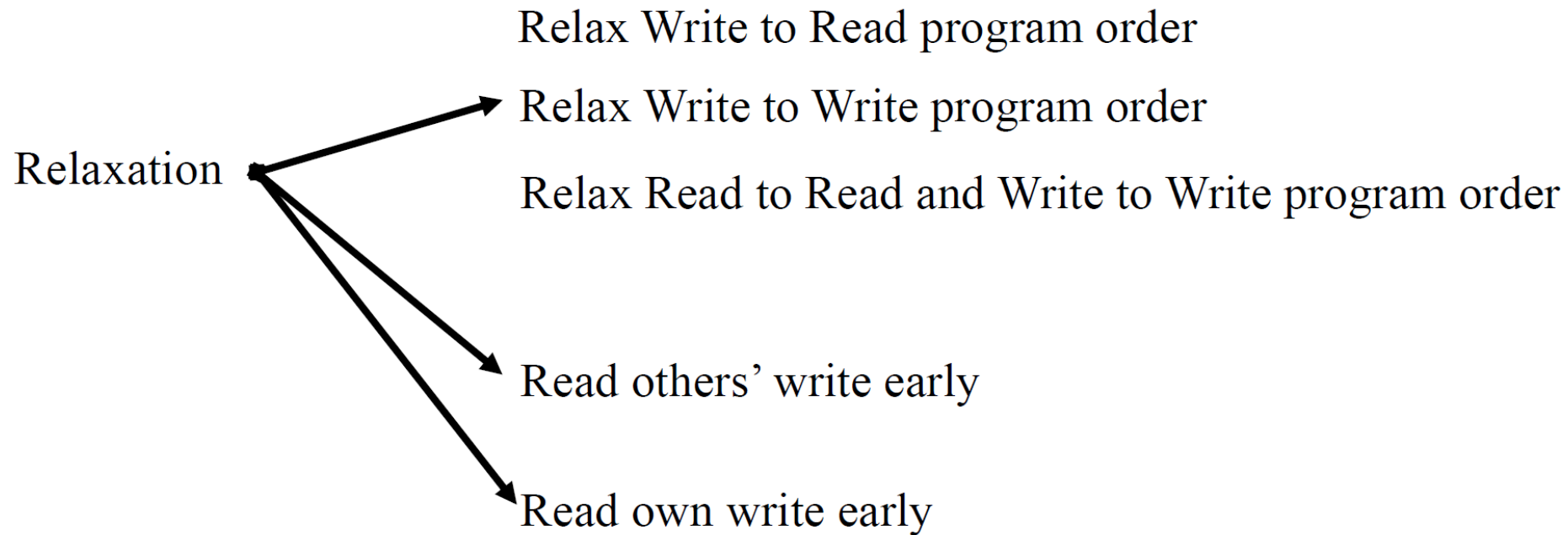
- Program ordering
- Store atomicity

This Lecture

- Relaxed consistency
- Data-race-free
- DeNovo coherence

Relaxed Consistency

S. V. Adve and K. Gharachorloo, *Shared Memory Consistency Models: A Tutorial*, Western Research Laboratory Technical Report, 1995.



Relaxing Write to Read Program Order

With store atomicity:

- e.g., IBM 370
- Example hardware optimizations: allow independent loads to bypass stores

Relaxing Write to Read Program Order

With store atomicity:

- e.g., IBM 370
- Example hardware optimizations: allow independent loads to bypass stores

```
// init
*x = 0; *y = 0;
```

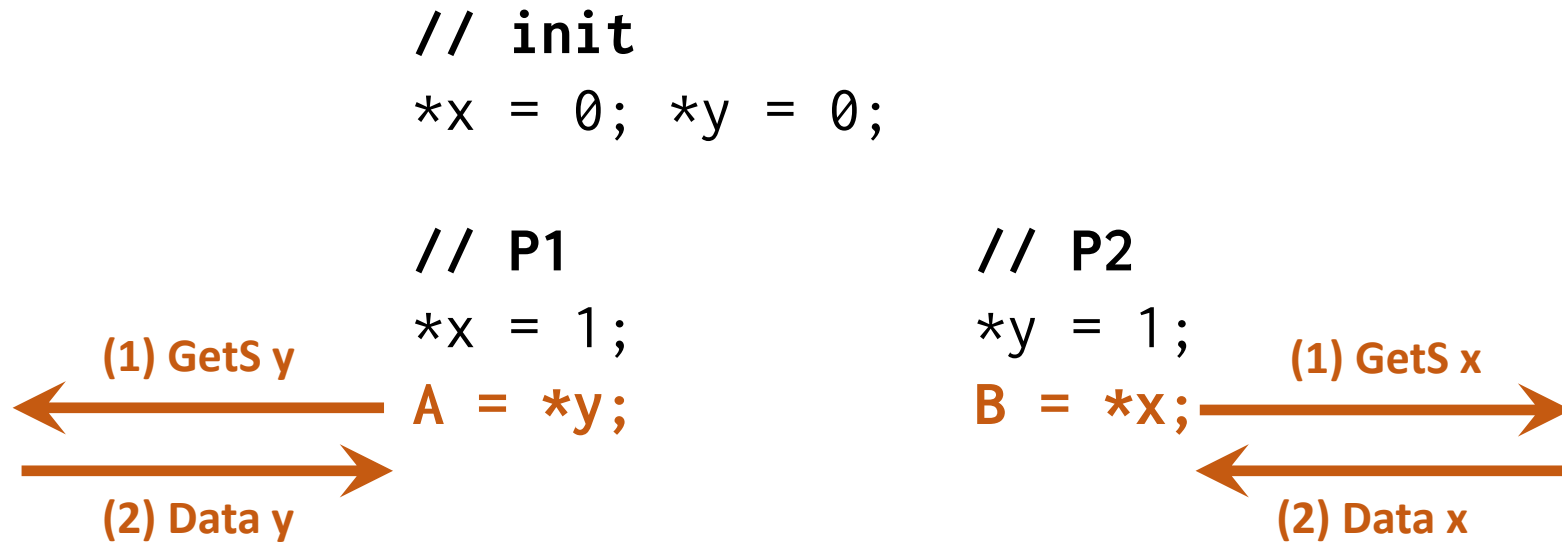
```
// P1
*x = 1;
A = *y;
```

```
// P2
*y = 1;
B = *x;
```

Relaxing Write to Read Program Order

With store atomicity:

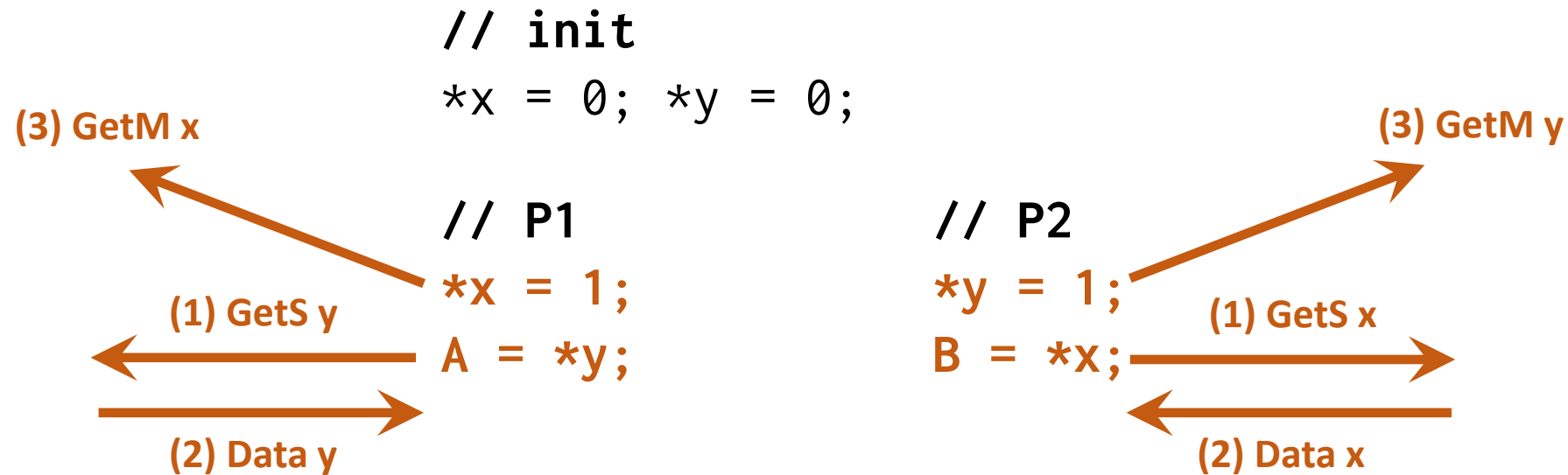
- e.g., IBM 370
- Example hardware optimizations: allow independent loads to bypass stores



Relaxing Write to Read Program Order

With store atomicity:

- e.g., IBM 370
- Example hardware optimizations: allow independent loads to bypass stores



Relaxing Write to Read Program Order

With store atomicity, but allow reading own write early:

- e.g., x86, SPARC V8 total store ordering (TSO)
- Example hardware optimizations: allow independent loads to bypass stores + store buffer with store-to-load forwarding

Relaxing Write to Read Program Order

With store atomicity, but allow reading own write early:

- e.g., x86, SPARC V8 total store ordering (TSO)
- Example hardware optimizations: allow independent loads to bypass stores + store buffer with store-to-load forwarding
- e.g., A=1 B=1 C=0 D=0 impossible

```
// init
```

```
*x = 0; *y = 0;
```

```
// P1
```

```
*x = 1;
```

```
A = *x;
```

```
C = *y;
```

```
// P2
```

```
B = *y;
```

```
D = *x;
```

```
// P3
```

```
*y = 1;
```

RAW dependence

read to read



Relaxing Write to Read Program Order

With store atomicity, but allow reading own write early:

- e.g., x86, SPARC V8 total store ordering (TSO)
- Example hardware optimizations: allow independent loads to bypass stores + store buffer with store-to-load forwarding
- e.g., A=1 B=1 C=0 D=0 impossible

```
// init
```

```
*x = 0; *y = 0;
```

```
// P1
```

```
*x = 1;
```

```
A = *x;
```

```
C = *y;
```

```
// P2
```

```
B = *y;
```

```
D = *x;
```

```
// P3
```

```
*y = 1;
```

store buffer

*x = 1

(1) GetM x



Relaxing Write to Read Program Order

With store atomicity, but allow reading own write early:

- e.g., x86, SPARC V8 total store ordering (TSO)
- Example hardware optimizations: allow independent loads to bypass stores + store buffer with store-to-load forwarding
- e.g., A=1 B=1 C=0 D=0 impossible

```
// init
```

```
*x = 0; *y = 0;
```

```
// P1
```

```
*x = 1;
```

```
A = *x;
```

```
C = *y;
```

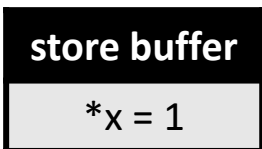
```
// P2
```

```
B = *y;
```

```
D = *x;
```

```
// P3
```

```
*y = 1;
```



(2) Data x

(1) GetM x



Relaxing Write to Read Program Order

With store atomicity, but allow reading own write early:

- e.g., x86, SPARC V8 total store ordering (TSO)
- Example hardware optimizations: allow independent loads to bypass stores + store buffer with store-to-load forwarding
- e.g., A=1 B=1 C=0 D=0 impossible

```
// init
```

```
*x = 0; *y = 0;
```

```
// P1
```

```
*x = 1;
```

```
A = *x;
```

```
C = *y;
```

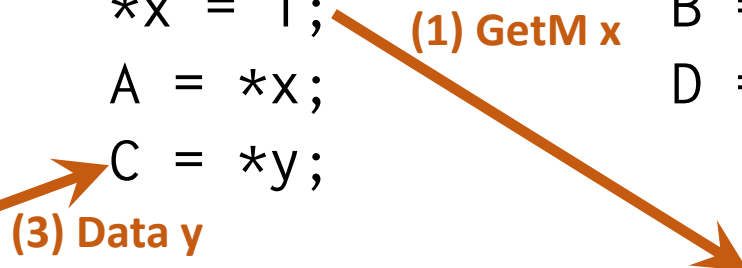
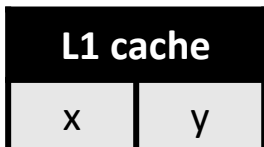
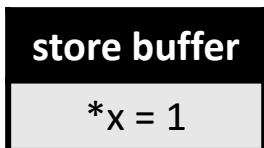
```
// P2
```

```
B = *y;
```

```
D = *x;
```

```
// P3
```

```
*y = 1;
```



Relaxing Write to Read Program Order

With store atomicity, but allow reading own write early:

- e.g., x86, SPARC V8 total store ordering (TSO)
- Example hardware optimizations: allow independent loads to bypass stores + store buffer with store-to-load forwarding
- e.g., A=1 B=1 C=0 D=0 **possible if can read own write early**

```
// init
```

```
*x = 0; *y = 0;
```

```
// P1
```

```
*x = 1;
```

```
A = *x;
```

```
C = *y;
```

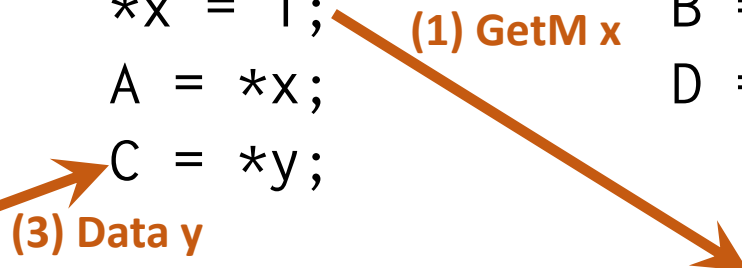
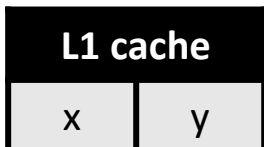
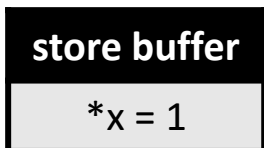
```
// P2
```

```
B = *y;
```

```
D = *x;
```

```
// P3
```

```
*y = 1;
```



Relaxing Write to Read Program Order

Without store atomicity (i.e., allow reading others' write early):

- e.g., Processor consistency (PC)
- Example hardware optimizations: allow data forwarding before all coherence invalidations

Relaxing Write to Read Program Order

Without store atomicity (i.e., allow reading others' write early):

- e.g., Processor consistency (PC)
- Example hardware optimizations: allow data forwarding before all coherence invalidations

```
// init
```

```
*x = 0; *y = 0;
```

```
// P1
```

```
*x = 1;
```

```
// P2
```

```
while(*x == 0);
```

```
*y = 1;
```

```
// P3
```

```
while(*y == 0);
```

```
A = *x;
```



Relaxing Write to Read Program Order

Without store atomicity (i.e., allow reading others' write early):

- e.g., Processor consistency (PC)
- Example hardware optimizations: allow data forwarding before all coherence invalidations

```
// init
```

```
*x = 0; *y = 0;
```

```
// P1
```

```
*x = 1;
```

```
// P2
```

```
while(*x == 0);  
*y = 1;
```

```
// P3
```

```
while(*y == 0);  
A = *x;
```

what if Inv x gets to P2
much earlier than P3?

(2) Inv x

directory x

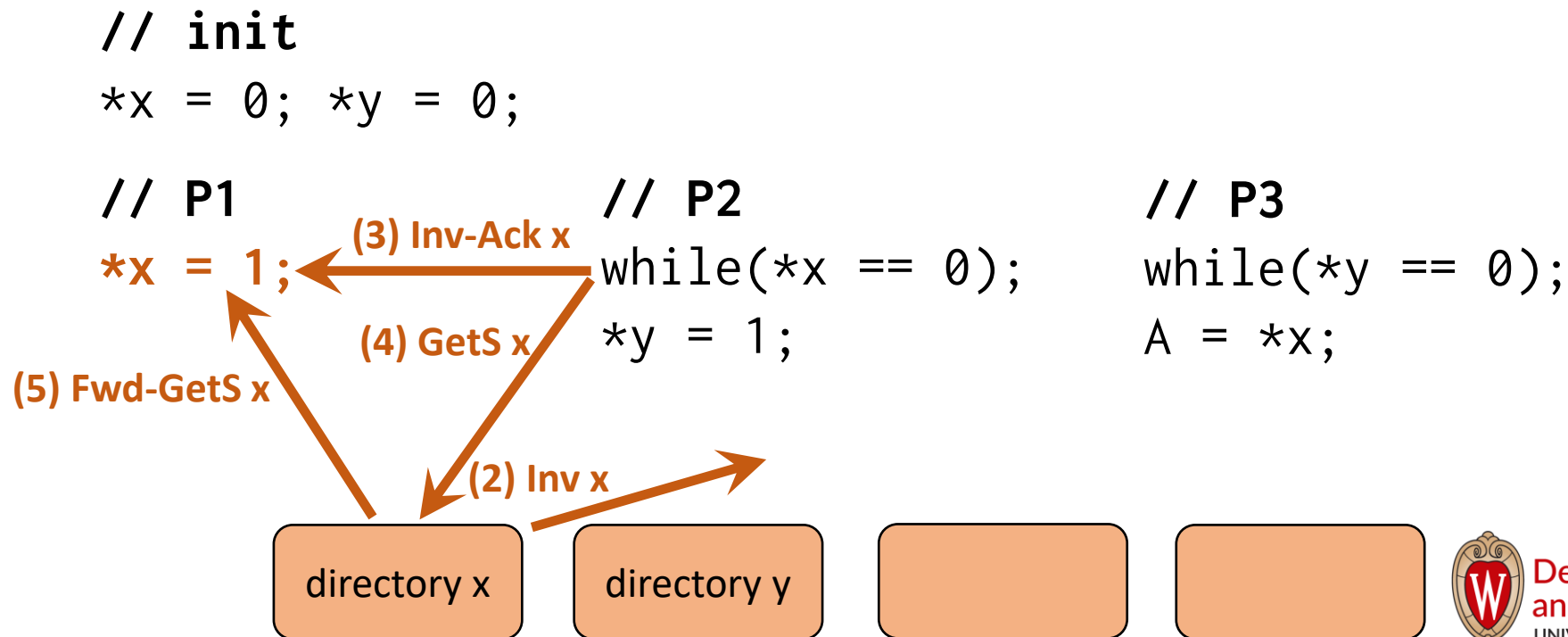
directory y



Relaxing Write to Read Program Order

Without store atomicity (i.e., allow reading others' write early):

- e.g., Processor consistency (PC)
- Example hardware optimizations: allow data forwarding before all coherence invalidations



Relaxing Write to Read Program Order

Cache Controller

Current state	Load	Store	Evict	Fwd-GetS	Fwd-GetM	Inv	Put-Ack	Data from dir [ack=0]	Data from dir [ack>0]	Data from owner	Inv-Ack	Inv-Ack (last)
I	GetS to dir → IS ^D	GetM to dir → IM ^{AD}										
IS ^D	stall	stall	stall			stall		→ S		→ S		
IM ^{AD}	stall	stall	stall	stall	stall			→ M	→ IM ^A	→ M	ack-- → IM ^{AD}	
IM ^A	stall	stall	stall	stall	stall						ack-- → IM ^A	→ M
S	hit → S	GetM to dir → SM ^{AD}	PutS to dir → SI ^A			Inv-Ack to req → I						
SM ^{AD}	hit → SM ^{AD}	stall	stall	stall	stall	Inv-Ack to req → IM ^{AD}		→ M	→ SM ^A		ack-- → SM ^{AD}	
SM ^A	hit → SM ^A	stall	stall	stall	stall						ack-- → SM ^A	→ M
M	hit → M	hit → M	PutM to dir → MI ^A	Data to req+dir → S	Data to req → I							
MI ^A	stall	stall	stall	Data to req+dir → SI ^A	Data to req → II ^A		→ I					
SI ^A	stall	stall	stall			Inv-Ack to req → II ^A	→ I					
II ^A	stall	stall	stall				→ I					

Relaxing Write to Read Program Order

Cache Controller

Current state	Load	Store	Evict	Fwd-GetS	Fwd-GetM	Inv	Put-Ack	Data from dir [ack=0]	Data from dir [ack>0]	Data from owner	Inv-Ack	Inv-Ack (last)
I	GetS to dir → IS ^D	GetM to dir → IM ^{AD}										
IS ^D	stall	stall	stall			stall		→ S		→ S		
IM ^{AD}	stall	stall	stall	stall	stall			→ M	→ IM ^A	→ M	ack-- → IM ^{AD}	
IM ^A	stall	stall	stall	stall	stall						ack-- → IM ^A	→ M
S	hit → S	GetM to dir → SM ^{AD}	PutS to dir → SI ^A			Inv-Ack to req → I						
SM ^{AD}	hit → SM ^{AD}	stall	stall	stall	stall	Inv-Ack to req → IM ^{AD}		→ M	→ SM ^A		ack-- → SM ^{AD}	
SM ^A	hit → SM ^A	stall	stall	stall	stall						ack-- → SM ^A	→ M
M	hit → M	hit → M	PutM to dir → MI ^A	Data to req+dir → S	Data to req → I							
MI ^A	stall	stall	stall	Data to req+dir → SI ^A	Data to req → II ^A		→ I					
SI ^A	stall	stall	stall			Inv-Ack to req → II ^A	→ I					
II ^A	stall	stall	stall				→ I					

Relaxing Write to Read Program Order

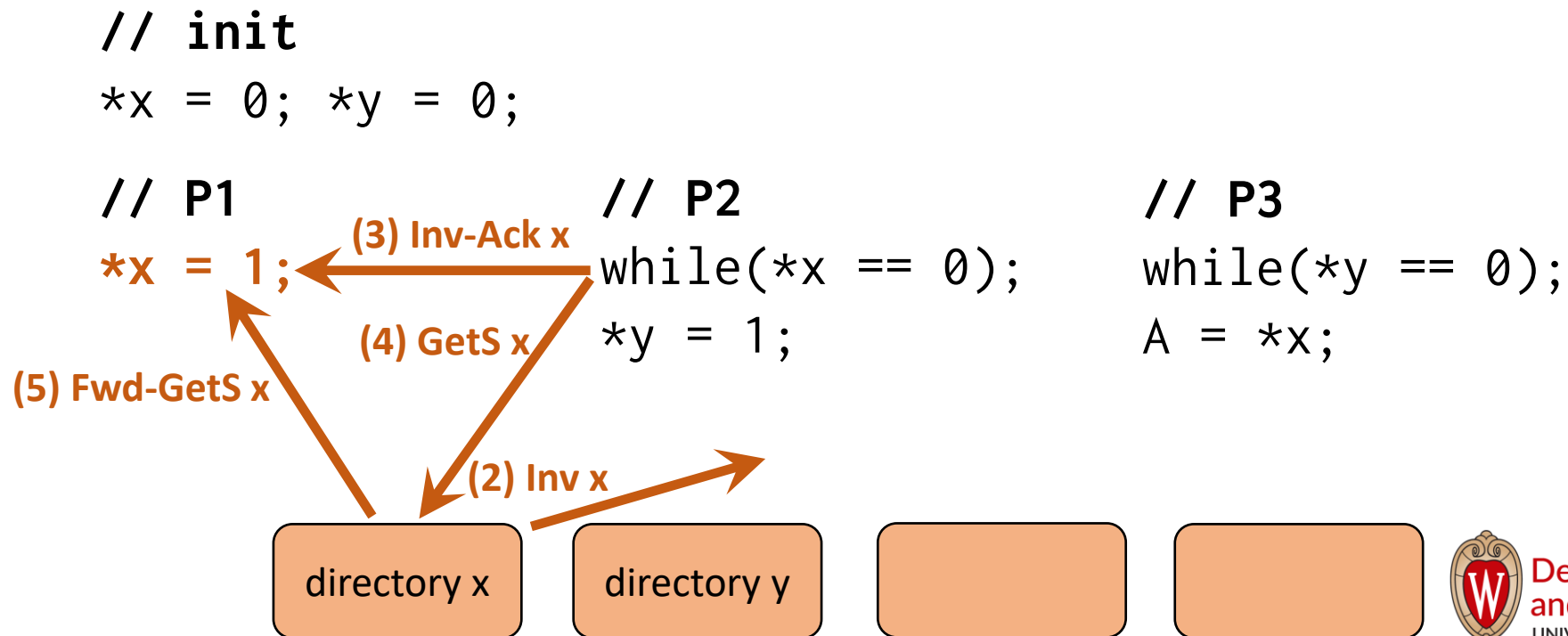
Cache Controller

Current state	Load	Store	Evict	Fwd-GetS	Fwd-GetM	Inv	Put-Ack	Data from dir [ack=0]	Data from dir [ack>0]	Data from owner	Inv-Ack	Inv-Ack (last)
I	GetS to dir → IS ^D	GetM to dir → IM ^{AD}										
IS ^D	stall	stall	stall			stall		→ S		→ S		
IM ^{AD}	stall	stall	stall	stall	stall			→ M	→ IM ^A	→ M	ack-- → IM ^{AD}	
IM ^A	stall	stall	stall	Data to req → ...	stall						ack-- → IM ^A	→ M
S	hit → S	GetM to dir → SM ^{AD}	PutS to dir → SI ^A			Inv-Ack to req → I						
SM ^{AD}	hit → SM ^{AD}	stall	stall	stall	stall	Inv-Ack to req → IM ^{AD}		→ M	→ SM ^A		ack-- → SM ^{AD}	
SM ^A	hit → SM ^A	stall	stall	stall	stall						ack-- → SM ^A	→ M
M	hit → M	hit → M	PutM to dir → MI ^A	Data to req+dir → S	Data to req → I							
MI ^A	stall	stall	stall	Data to req+dir → SI ^A	Data to req → II ^A		→ I					
SI ^A	stall	stall	stall			Inv-Ack to req → II ^A	→ I					
II ^A	stall	stall	stall				→ I					

Relaxing Write to Read Program Order

Without store atomicity (i.e., allow reading others' write early):

- e.g., Processor consistency (PC)
- Example hardware optimizations: allow data forwarding before all coherence invalidations



Relaxing Write to Write Program Order

With relaxed write to read:

- e.g., SPARC V8 partial store ordering (PSO)
- Example hardware optimizations: retire stores out-of-order, allow multiple in-flight store misses

Relaxing Write to Write Program Order

With relaxed write to read:

- e.g., SPARC V8 partial store ordering (PSO)
- Example hardware optimizations: retire stores out-of-order, allow multiple in-flight store misses

```
// init
```

```
*x = 0; *y = 0;
```

```
// P1
```

```
*x = 1;
```

```
*y = 1;
```

```
// P2
```

```
if(*y != 0) {
```

```
    A = *x;
```

```
    assert(A != 0);
```

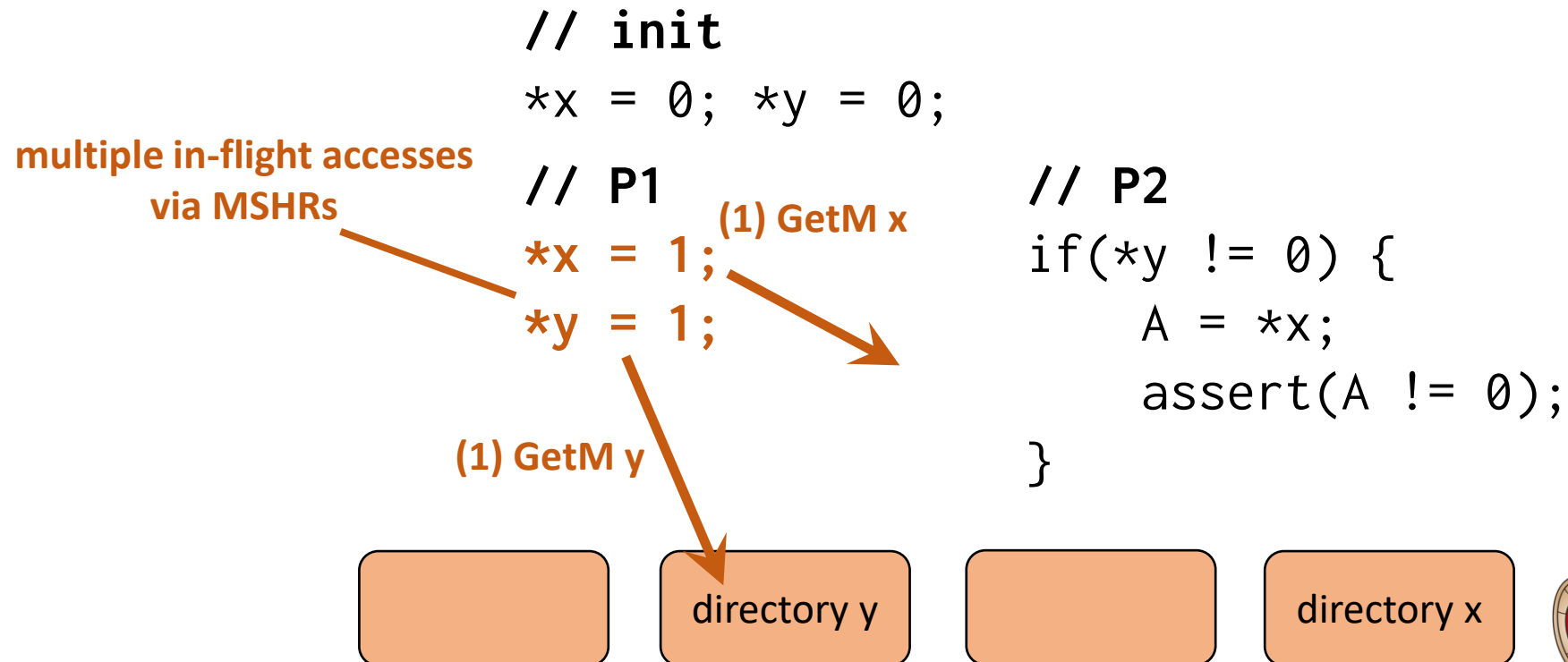
```
}
```



Relaxing Write to Write Program Order

With relaxed write to read:

- e.g., SPARC V8 partial store ordering (PSO)
- Example hardware optimizations: retire stores out-of-order, allow multiple in-flight store misses



Relaxing All Program Orders

With store atomicity:

- e.g., Alpha, SPARC V9 relaxed memory order (RMO)

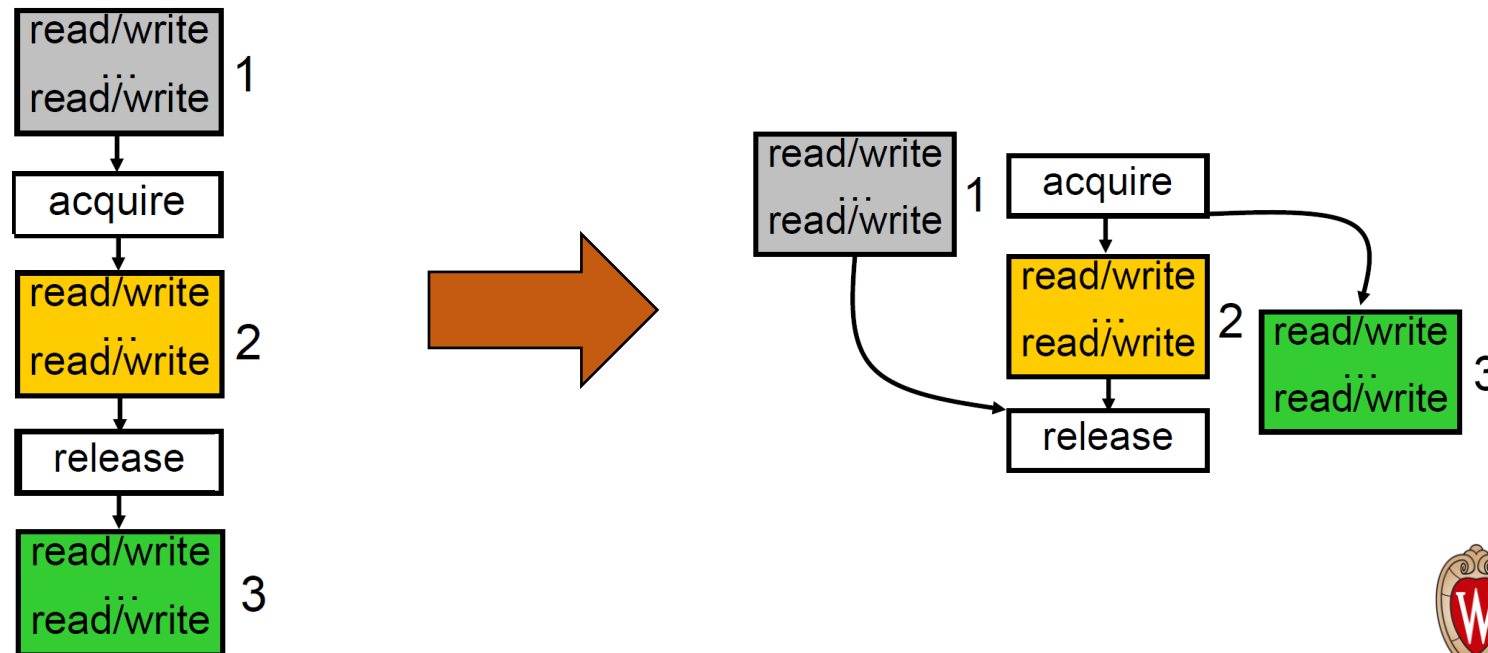
Without store atomicity:

- e.g., IBM PowerPC

Relaxing All Program Orders

Release consistency (RC):

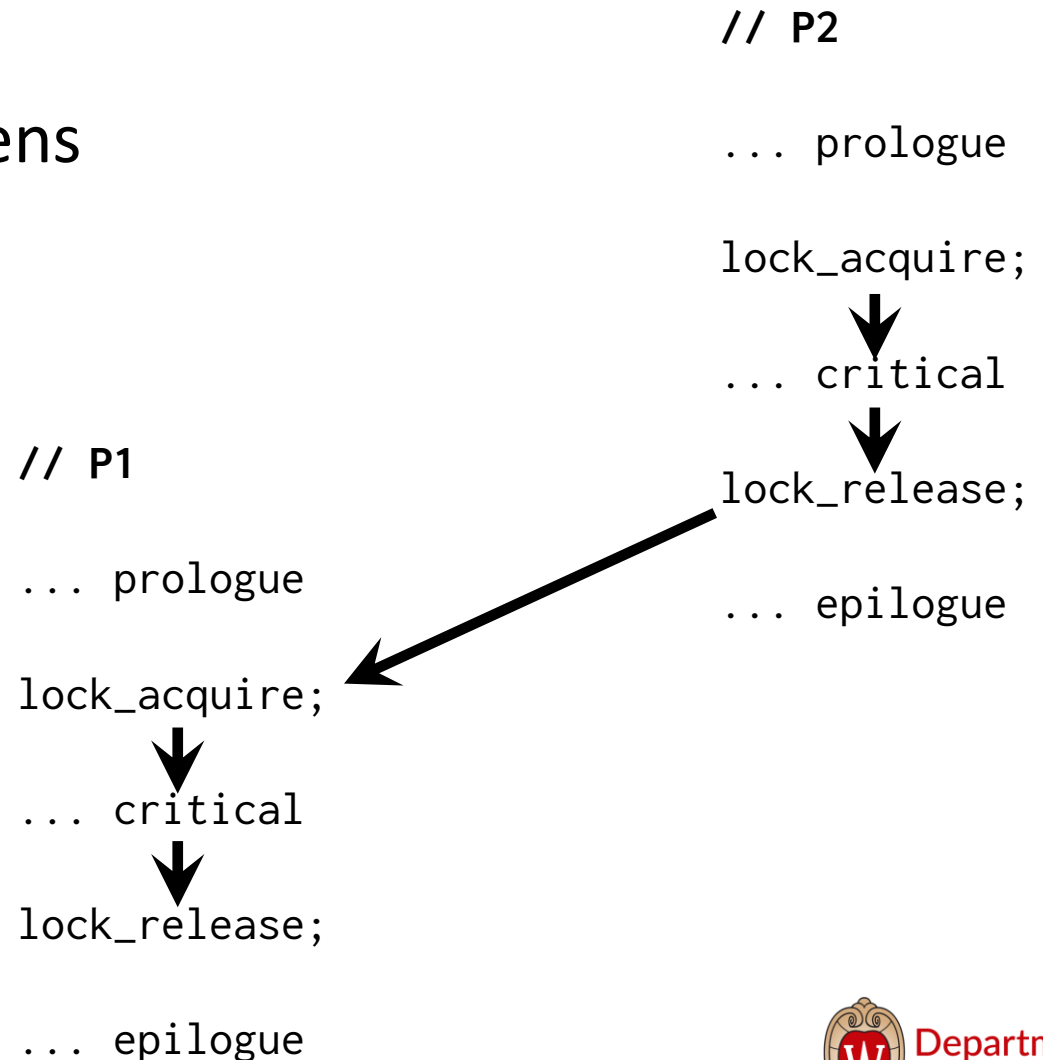
- Maintain sequential consistency only within lock acquire-release
- ISA explicitly recognizes acquire-release (e.g., atomic-read-write)
- Enforce ordering between 1) acquire and everything after it, 2) release and everything before it



Relaxing All Program Orders

Release consistency (RC):

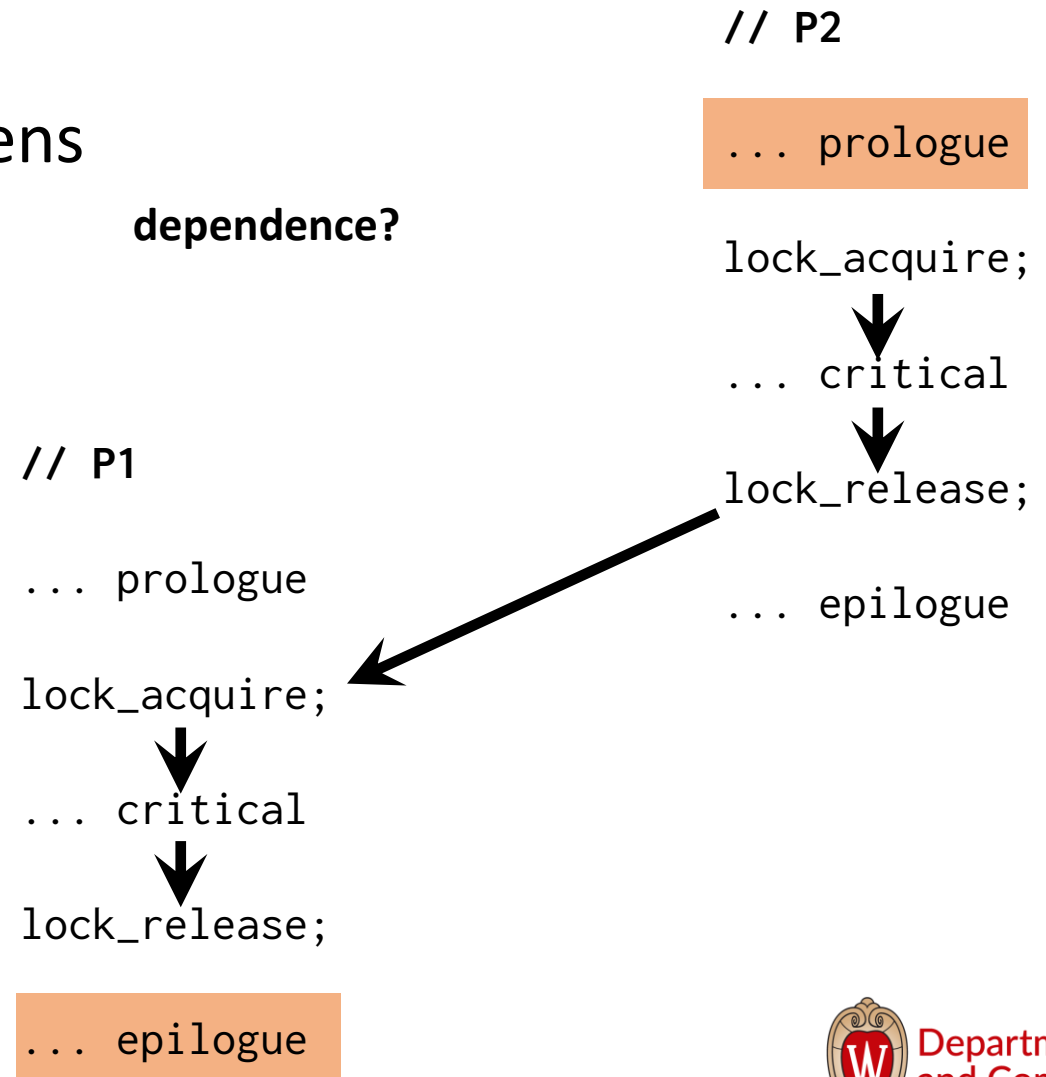
- e.g., If P2's critical section happens before P1's critical section



Relaxing All Program Orders

Release consistency (RC):

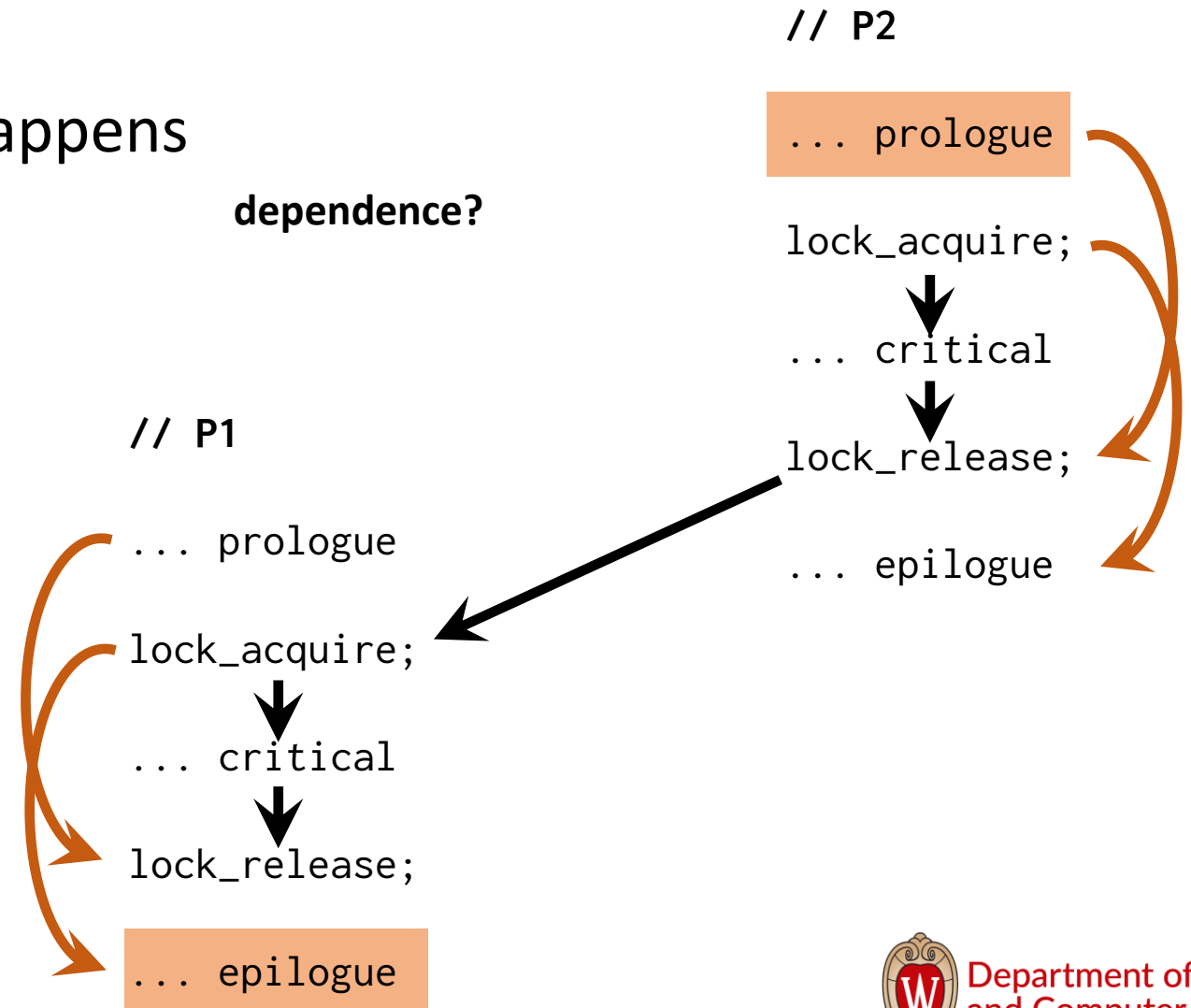
- e.g., If P2's critical section happens before P1's critical section



Relaxing All Program Orders

Release consistency (RC):

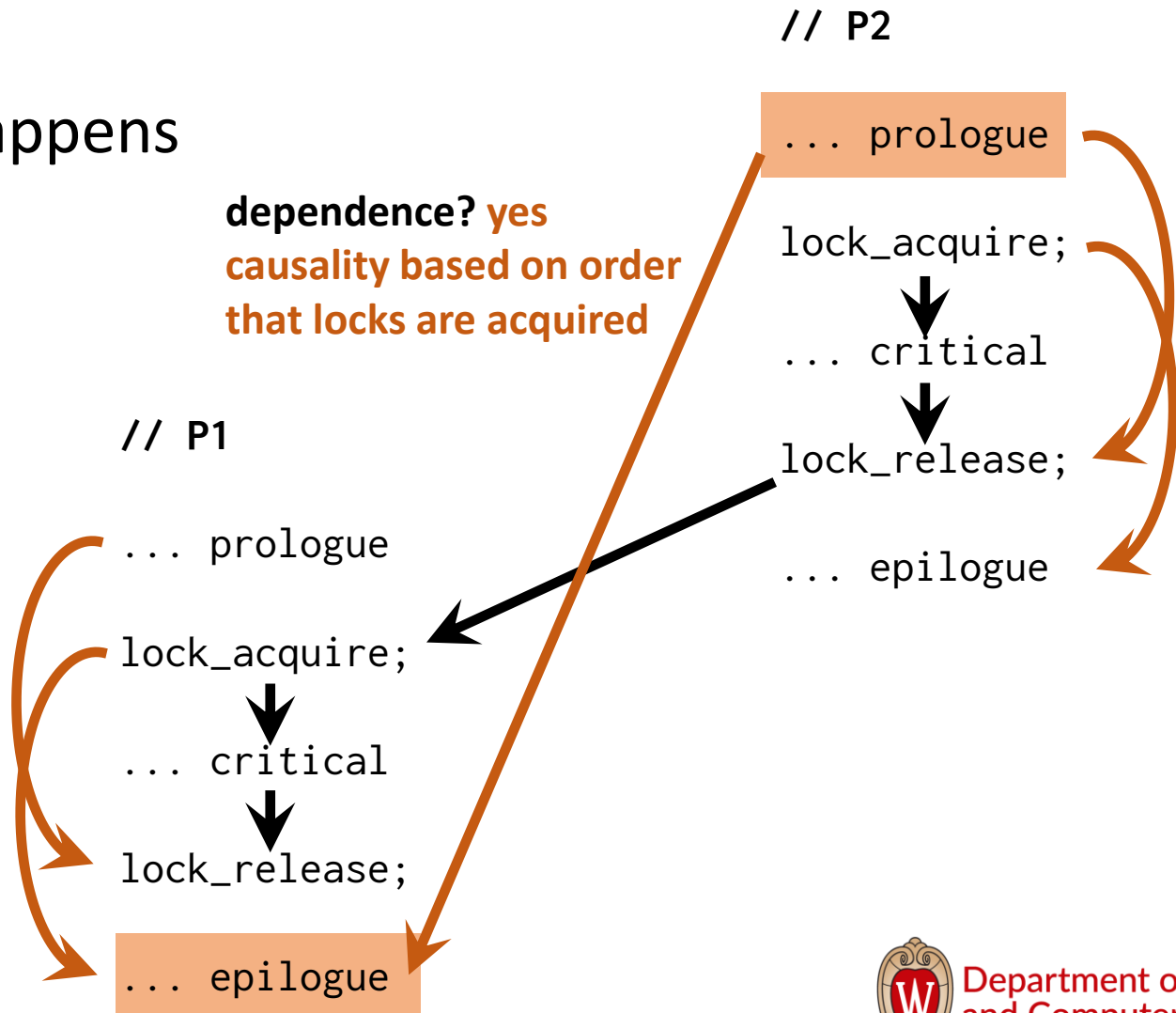
- e.g., If P2's critical section happens before P1's critical section



Relaxing All Program Orders

Release consistency (RC):

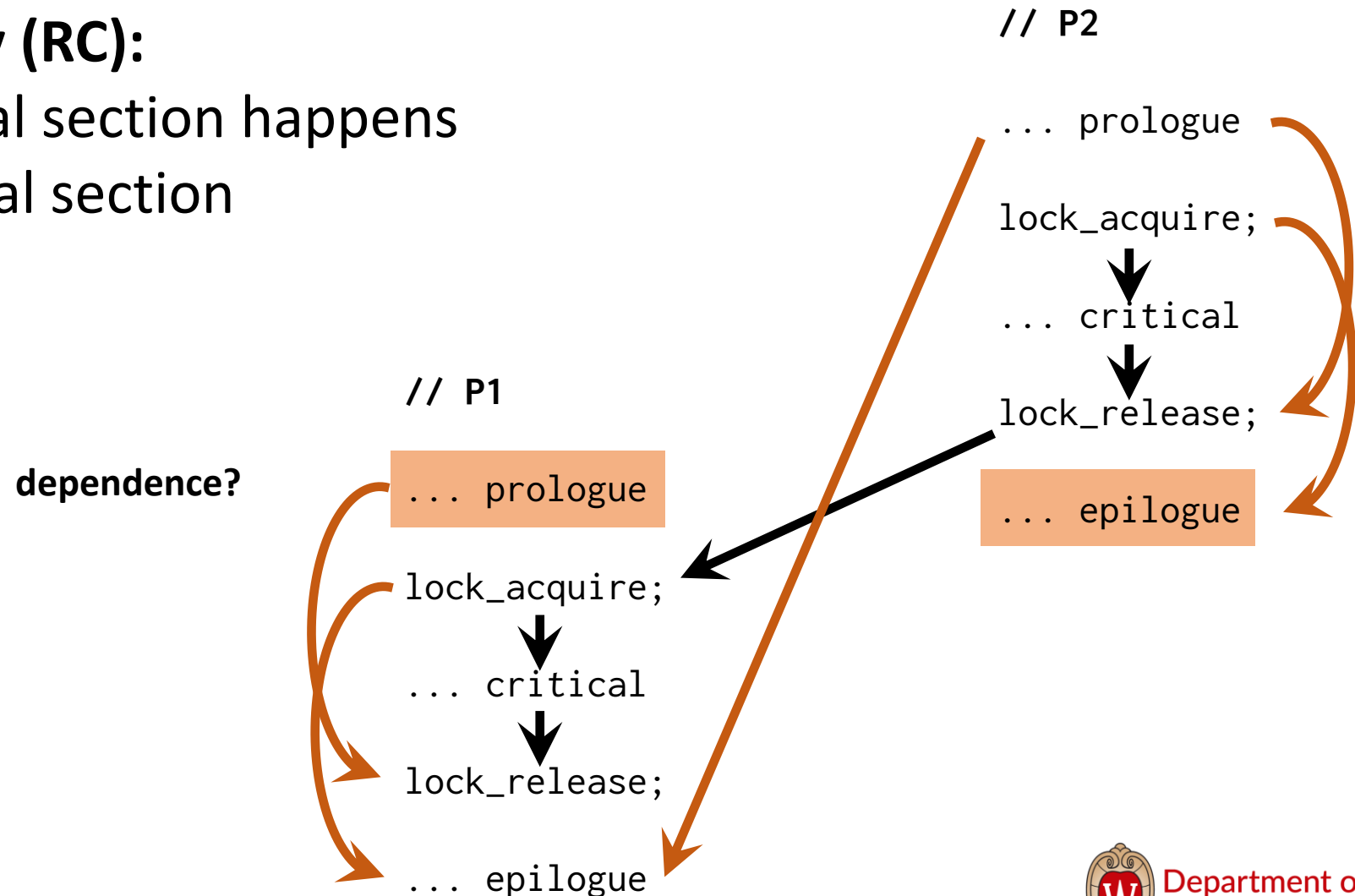
- e.g., If P2's critical section happens before P1's critical section



Relaxing All Program Orders

Release consistency (RC):

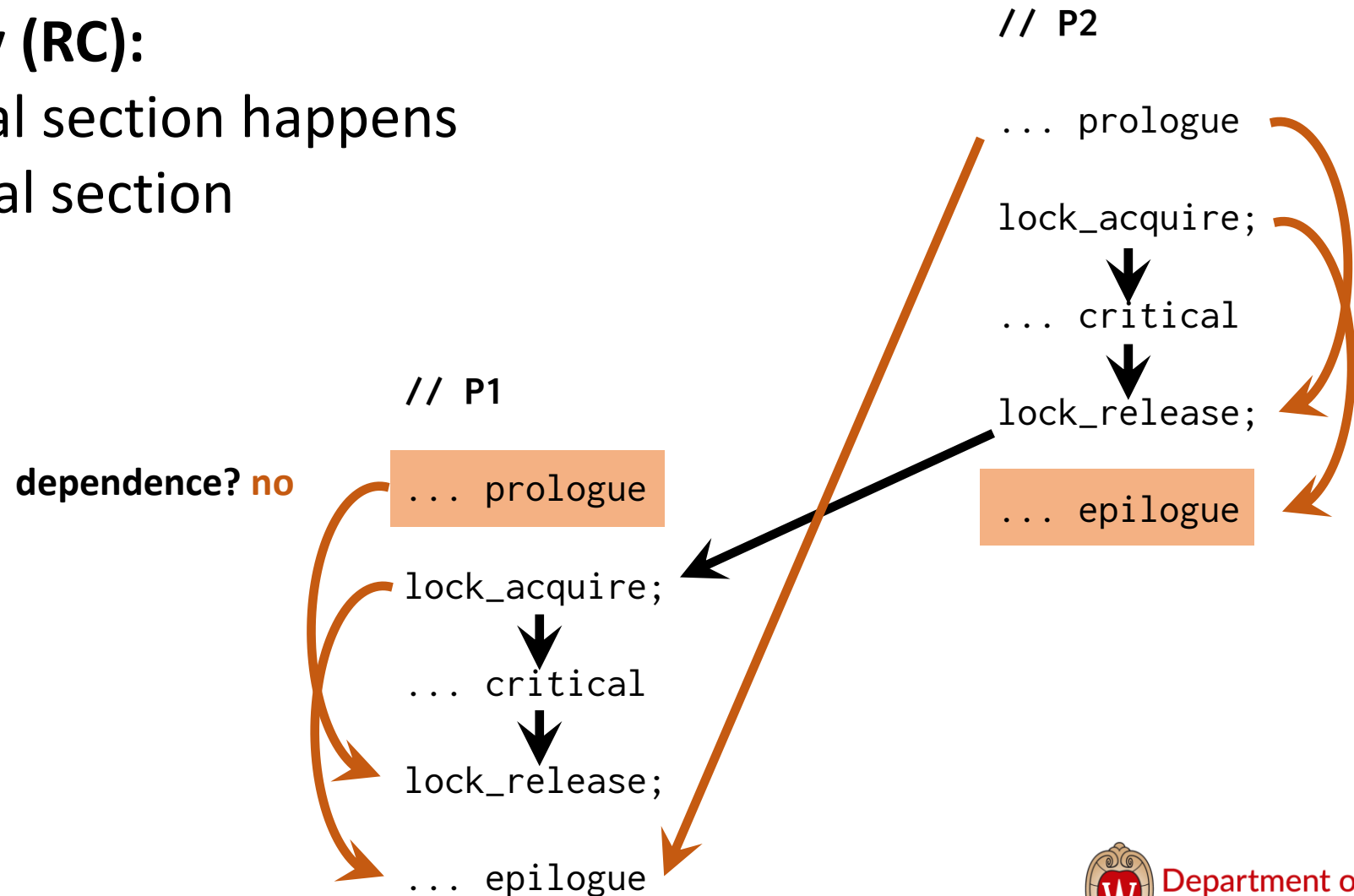
- e.g., If P2's critical section happens before P1's critical section



Relaxing All Program Orders

Release consistency (RC):

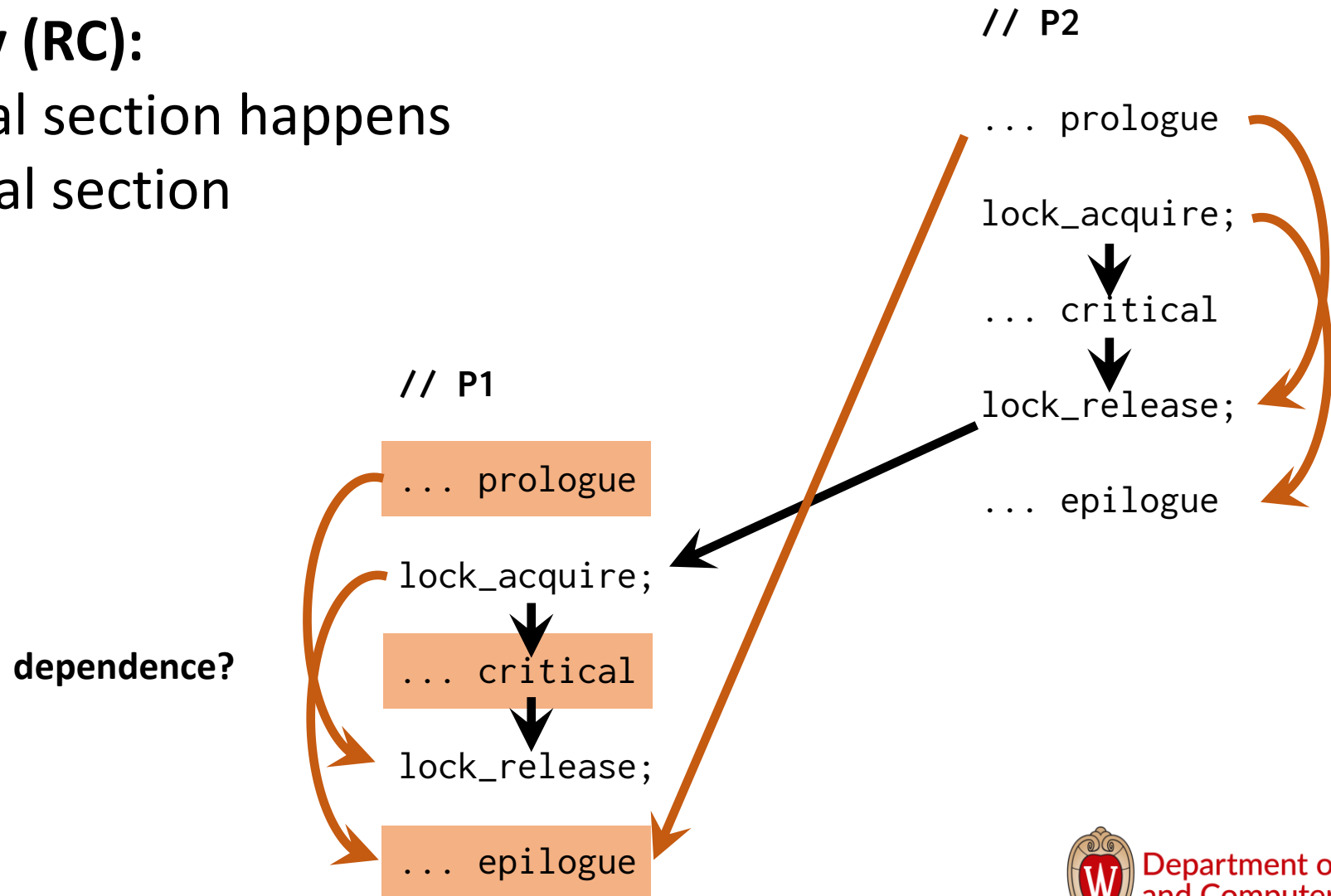
- e.g., If P2's critical section happens before P1's critical section



Relaxing All Program Orders

Release consistency (RC):

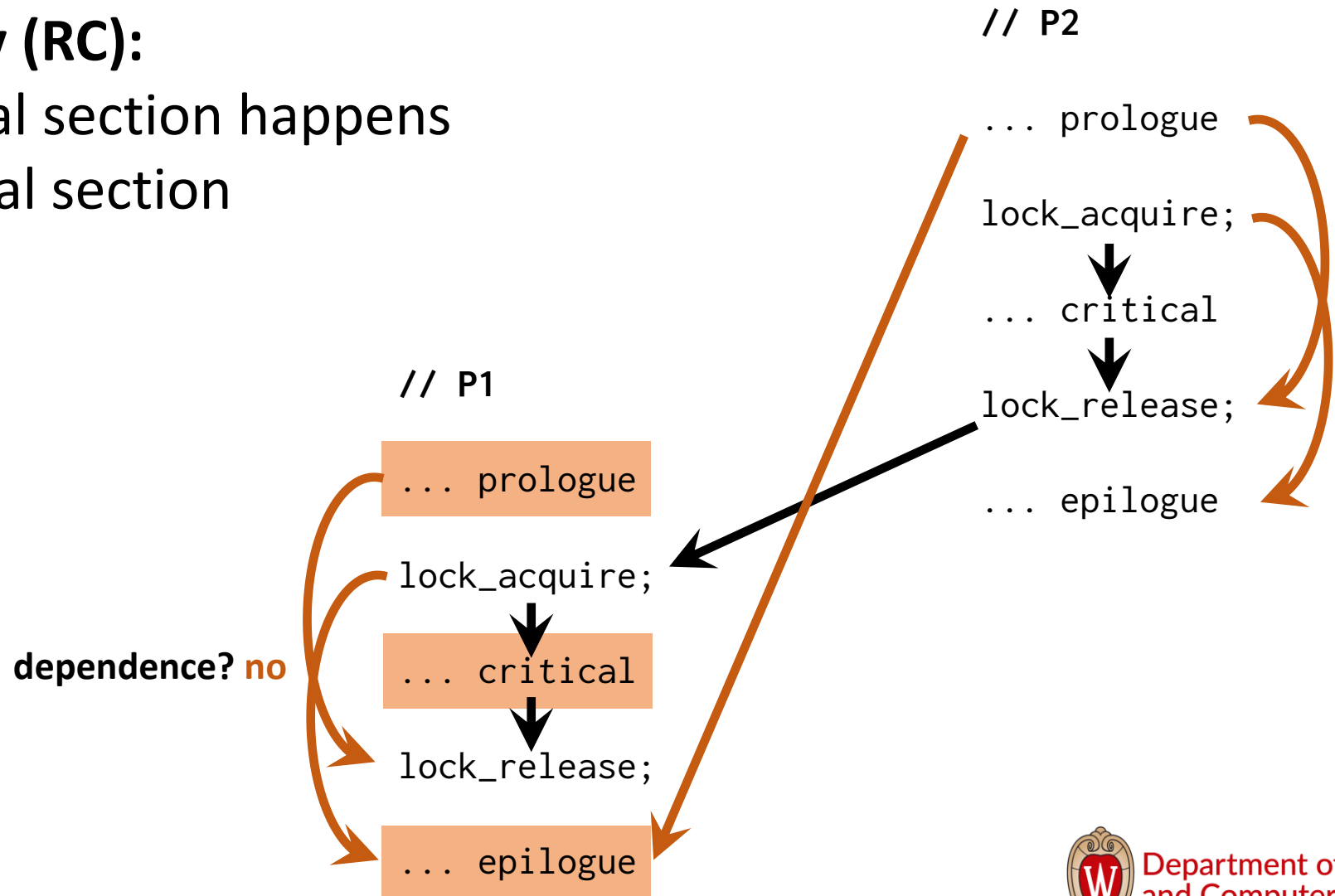
- e.g., If P2's critical section happens before P1's critical section



Relaxing All Program Orders

Release consistency (RC):

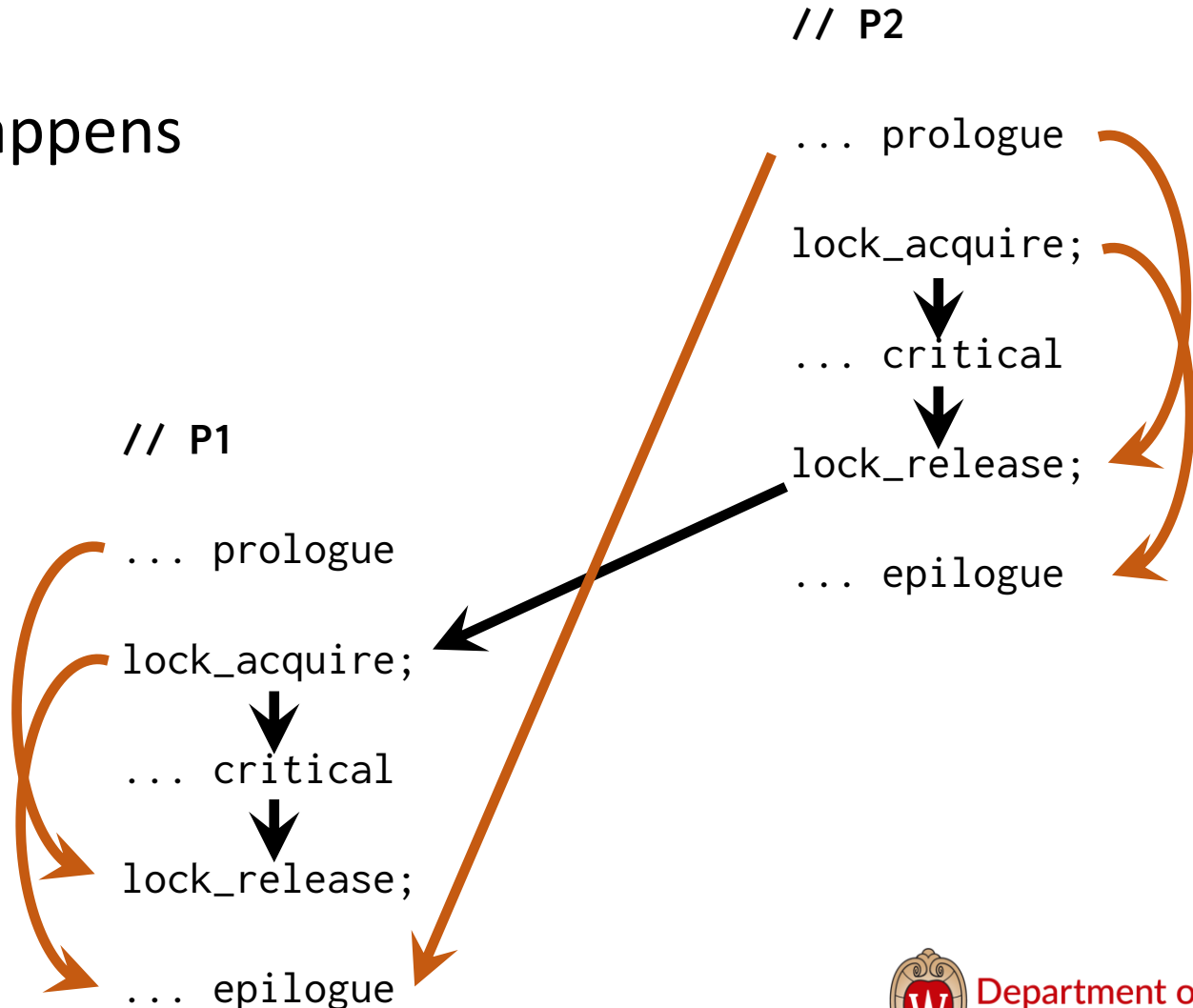
- e.g., If P2's critical section happens before P1's critical section



Relaxing All Program Orders

Release consistency (RC):

- e.g., If P2's critical section happens before P1's critical section



Data-Race-Free

Two memory accesses form a **race** if:

- Access same memory location, and at least one is a write
- From different threads and can be executed in parallel
- **Data race**: a race for memory location that contains data

```
data race // P1
*x = 757;
*flag = 1;
```

```
// P2
if(*flag != 0) {
    A = *x;
    assert(A != 0);
}
```

Data-Race-Free

A program is **data-race-free (DRF)** if no data races under SC execution:

- [S. V. Adve and M. D. Hill, *Weak Ordering - A New Definition*, ISCA, 1990]
- The only races are for synchronization variables
 - System enforces ordering only for explicitly-declared synch variables (i.e., atomics)
 - HW can then employ relaxed consistency for everything else

must explicitly declare
synchronization

```
// P1
*x = 757;
*flag = 1;
```

```
// P2
if(*flag != 0) {
    A = *x;
    assert(A != 0);
}
```

Data-Race-Free

Most straightforward way to write a data-race-free program is to put all (potentially conflicting) shared accesses within critical sections

- Memory locations accessed outside critical sections are either private or read-only

```
// P1
```

```
...
```

```
lock();
```

```
*x = *x + 1;
```

```
unlock();
```

```
...
```

```
// P2
```

```
...
```

```
lock();
```

```
*y = *x;
```

```
unlock();
```

```
...
```



Data-Race-Free

Generally, a program is data-race-free if threads are synchronized well

➤ e.g., Is this program data-race-free?

```
// init
```

```
*x = 0; *y = 0;
```

```
// P1
```

```
*x = 1;
```

```
lock();
```

```
*y = 1;
```

```
unlock();
```

```
// P2
```

```
lock();
```

```
A = *y;
```

```
unlock();
```

```
B = *x;
```

Data-Race-Free

Generally, a program is data-race-free if threads are synchronized well

➤ e.g., Is this program data-race-free?

```
// init
```

```
*x = 0; *y = 0;
```

```
// P1
```

```
*x = 1;
```

```
lock();
```

```
*y = 1;
```

```
unlock();
```

```
// P2
```

```
lock();
```

```
A = *y;
```

```
unlock();
```

```
B = *x;
```

if P1 locks first:



Data-Race-Free

Generally, a program is data-race-free if threads are synchronized well

➤ e.g., Is this program data-race-free?

```
// init
```

```
*x = 0; *y = 0;
```

```
// P1
```

```
*x = 1;
```

```
lock();
```

```
*y = 1;
```

```
unlock();
```

```
// P2
```

```
lock();
```

```
A = *y;
```

```
unlock();
```

```
B = *x;
```

RAW
RAW

if P1 locks first: **no race**



Data-Race-Free

Generally, a program is data-race-free if threads are synchronized well

➤ e.g., Is this program data-race-free?

```
// init
```

```
*x = 0; *y = 0;
```

```
// P1
```

```
*x = 1;
```

```
lock();
```

```
*y = 1;
```

```
unlock();
```

```
// P2
```

```
lock();
```

```
A = *y;
```

```
unlock();
```

```
B = *x;
```

if P1 locks first: **no race**

if P2 locks first:



Data-Race-Free

Generally, a program is data-race-free if threads are synchronized well

➤ e.g., Is this program data-race-free?

```
// init
```

```
*x = 0; *y = 0;
```

```
// P1
```

```
*x = 1;
```

```
lock();
```

```
*y = 1;
```

```
unlock();
```

```
// P2
```

```
lock();
```

```
A = *y;
```

```
unlock();
```

```
B = *x;
```

WAR

if P1 locks first: **no race**

if P2 locks first:



Data-Race-Free

Generally, a program is data-race-free if threads are synchronized well

➤ e.g., Is this program data-race-free? **No**

```
// init
```

```
*x = 0; *y = 0;
```

```
// P1
```

```
*x = 1;
```

```
lock();
```

```
*y = 1;
```

```
unlock();
```

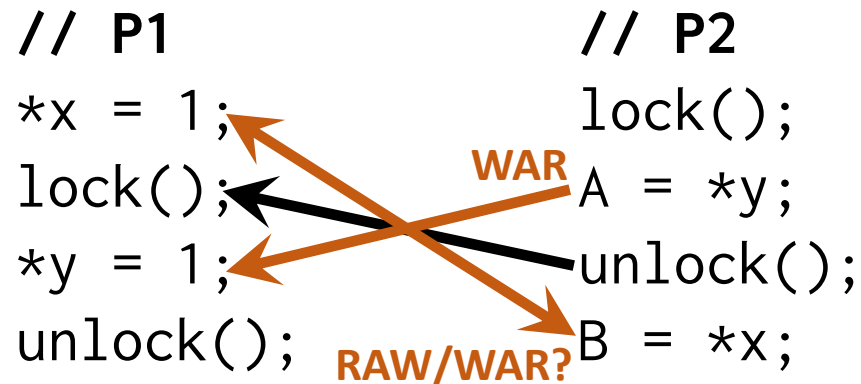
```
// P2
```

```
lock();
```

```
A = *y;
```

```
unlock();
```

```
B = *x;
```



if P1 locks first: **no race**

if P2 locks first: **race!**



Data-Race-Free

Generally, a program is data-race-free if threads are synchronized well

➤ e.g., Is this program data-race-free?

```
// init
```

```
*x = 0; *y = 0;
```

```
// P1
```

```
*x = 1;
```

```
lock();
```

```
*y = 1;
```

```
unlock();
```

```
// P2
```

```
lock();
```

```
A = *y;
```

```
unlock();
```

```
if(A) B = *x;
```

Data-Race-Free

Generally, a program is data-race-free if threads are synchronized well

➤ e.g., Is this program data-race-free?

```
// init
```

```
*x = 0; *y = 0;
```

```
// P1
```

```
*x = 1;
```

```
lock();
```

```
*y = 1;
```

```
unlock();
```

```
// P2
```

```
lock();
```

```
A = *y;
```

```
unlock();
```

```
if(A) B = *x;
```

if P1 locks first:

Data-Race-Free

Generally, a program is data-race-free if threads are synchronized well

➤ e.g., Is this program data-race-free?

```
// init
```

```
*x = 0; *y = 0;
```

```
// P1
```

```
*x = 1;
```

```
lock();
```

```
*y = 1;
```

```
unlock();
```

```
// P2
```

```
lock();
```

```
A = *y;
```

```
unlock();
```

```
if(A) B = *x;
```

RAW
RAW

if P1 locks first: **no race**



Data-Race-Free

Generally, a program is data-race-free if threads are synchronized well

➤ e.g., Is this program data-race-free?

```
// init
```

```
*x = 0; *y = 0;
```

```
// P1
```

```
*x = 1;
```

```
lock();
```

```
*y = 1;
```

```
unlock();
```

```
// P2
```

```
lock();
```

```
A = *y;
```

```
unlock();
```

```
if(A) B = *x;
```

if P1 locks first: **no race**

if P2 locks first:



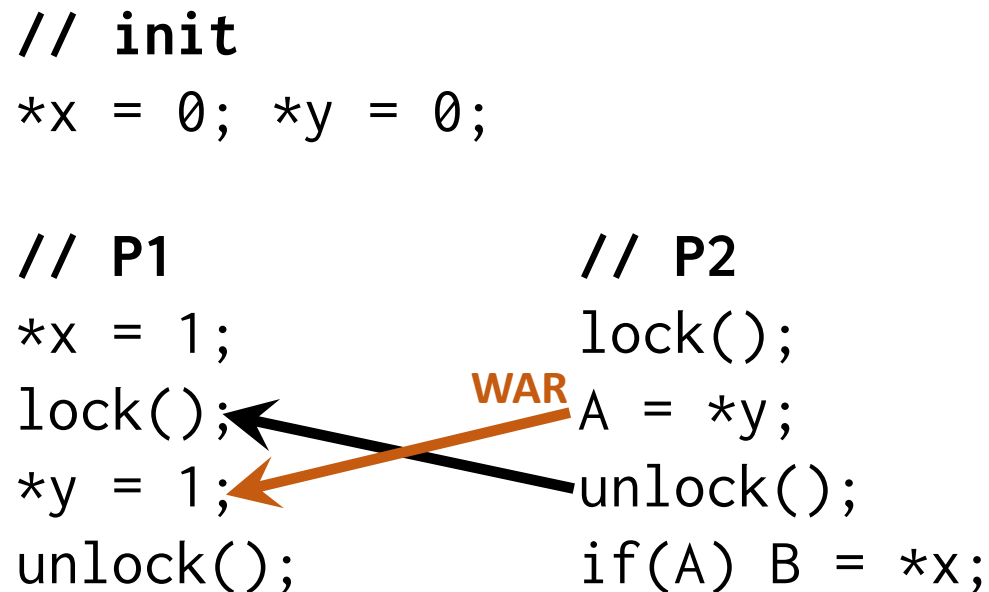
Data-Race-Free

Generally, a program is data-race-free if threads are synchronized well

➤ e.g., Is this program data-race-free? **Yes**

```
// init
*x = 0; *y = 0;

// P1                // P2
*x = 1;              lock();
lock();              A = *y;
*y = 1;              unlock();
unlock();             if(A) B = *x;
```



if P1 locks first: **no race**

if P2 locks first: **no race**

DeNovo

Can simplify cache coherence for data-race-free programs [PACT 2011]

- If program has predefined parallel phases (i.e., *disciplined parallelism*), caches can **self-invalidate** potentially stale data between phases
 - Keep any data you read in most recent phase or data you own; invalidate the rest
- Directory just tracks the owner; eliminates transient states

```
// P1
barrier();
    ... = A[];
barrier();
    B[] = C[];
barrier();
    ...
```

```
// P2
barrier();
    D[] = ...;
barrier();
    ... = A[];
barrier();
    ...
```

DeNovo

Can simplify cache coherence for data-race-free programs [PACT 2011]

- If program has predefined parallel phases (i.e., *disciplined parallelism*), caches can **self-invalidate** potentially stale data between phases
 - Keep any data you read in most recent phase or data you own; invalidate the rest
- Directory just tracks the owner; eliminates transient states

	<pre>// P1 barrier(); ... = A[]; barrier(); B[] = C[]; barrier(); ...</pre>	<pre>// P2 barrier(); D[] = ...; barrier(); ... = A[]; barrier(); ...</pre>
P1 invalidates:		

DeNovo

Can simplify cache coherence for data-race-free programs [PACT 2011]

- If program has predefined parallel phases (i.e., *disciplined parallelism*), caches can **self-invalidate** potentially stale data between phases
 - Keep any data you read in most recent phase or data you own; invalidate the rest
- Directory just tracks the owner; eliminates transient states

	<pre>// P1 barrier(); ... = A[]; barrier(); B[] = C[]; barrier(); ...</pre>	<pre>// P2 barrier(); D[] = ...; barrier(); ... = A[]; barrier(); ...</pre>
P1 invalidates: A		

DeNovo

Can simplify cache coherence for data-race-free programs [PACT 2011]

- If program has predefined parallel phases (i.e., *disciplined parallelism*), caches can **self-invalidate** potentially stale data between phases
 - Keep any data you read in most recent phase or data you own; invalidate the rest
- Directory just tracks the owner; eliminates transient states

	<pre>// P1 barrier(); ... = A[]; barrier(); B[] = C[]; barrier(); ...</pre>	<pre>// P2 barrier(); D[] = ...; barrier(); ... = A[]; barrier(); ...</pre>	
P1 invalidates: A	-----		P2 invalidates:

DeNovo

Can simplify cache coherence for data-race-free programs [PACT 2011]

- If program has predefined parallel phases (i.e., *disciplined parallelism*), caches can **self-invalidate** potentially stale data between phases
 - Keep any data you read in most recent phase or data you own; invalidate the rest
- Directory just tracks the owner; eliminates transient states

	<pre>// P1 barrier(); ... = A[]; barrier(); B[] = C[]; barrier(); ...</pre>	<pre>// P2 barrier(); D[] = ...; barrier(); ... = A[]; barrier(); ...</pre>	
P1 invalidates: A			P2 invalidates: –

DeNovo

Can simplify cache coherence for data-race-free programs [PACT 2011]

- If program has predefined parallel phases (i.e., *disciplined parallelism*), caches can **self-invalidate** potentially stale data between phases
 - Keep any data you read in most recent phase or data you own; invalidate the rest
- Directory just tracks the owner; eliminates transient states

	<pre>// P1 barrier(); ... = A[]; barrier(); B[] = C[]; barrier(); ...</pre>	<pre>// P2 barrier(); D[] = ...; barrier(); ... = A[]; barrier(); B[] = ...;</pre>	what if P2 writes B next?
P1 invalidates: A	-----		P2 invalidates: --

DeNovo

Can simplify cache coherence for data-race-free programs [PACT 2011]

- If program has predefined parallel phases (i.e., *disciplined parallelism*), caches can **self-invalidate** potentially stale data between phases
 - Keep any data you read in most recent phase or data you own; invalidate the rest
- Directory just tracks the owner; eliminates transient states

	<pre>// P1 barrier(); ... = A[]; barrier(); B[] = C[]; barrier(); ...</pre>	<pre>// P2 barrier(); D[] = ...; barrier(); ... = A[]; barrier(); B[] = ...;</pre>	what if P2 writes B next? Directory asks P1 to forward
P1 invalidates: A			P2 invalidates: -

DeNovo

Can simplify cache coherence for data-race-free programs [PACT 2011]

- If program has predefined parallel phases (i.e., *disciplined parallelism*), caches can **self-invalidate** potentially stale data between phases
 - Keep any data you read in most recent phase or data you own; invalidate the rest
- Directory just tracks the owner; eliminates transient states

	<pre>// P1 barrier(); ... = A[]; barrier(); B[] = C[]; barrier(); ...</pre>	<pre>// P2 barrier(); D[] = ...; barrier(); ... = A[]; barrier(); ...</pre>	why is it ok to keep C in P1 cache?
P1 invalidates: A	-----		P2 invalidates: --

DeNovo

Can simplify cache coherence for data-race-free programs [PACT 2011]

- If program has predefined parallel phases (i.e., *disciplined parallelism*), caches can **self-invalidate** potentially stale data between phases
 - Keep any data you read in most recent phase or data you own; invalidate the rest
- Directory just tracks the owner; eliminates transient states

	<pre>// P1 barrier(); ... = A[]; barrier(); B[] = C[]; barrier(); ... = C[];</pre>	<pre>// P2 barrier(); D[] = ...; barrier(); ... = A[]; barrier(); ...</pre>	why is it ok to keep C in P1 cache? if P1 reads C next, P2 is guaranteed to not write C
P1 invalidates: A	-----		P2 invalidates: --

DeNovo

Can simplify cache coherence for data-race-free programs [PACT 2011]

- If program has predefined parallel phases (i.e., *disciplined parallelism*), caches can **self-invalidate** potentially stale data between phases
 - Keep any data you read in most recent phase or data you own; invalidate the rest
- Directory just tracks the owner; eliminates transient states

	<pre>// P1 barrier(); ... = A[]; barrier(); B[] = C[]; barrier(); ...</pre>	<pre>// P2 barrier(); D[] = ...; barrier(); ... = A[]; barrier(); C[] = ...;</pre>	why is it ok to keep C in P1 cache? if P2 writes C next, P1 is guaranteed to not read C
P1 invalidates: A	-----		P2 invalidates: --

DeNovo

Can simplify cache coherence for data-race-free programs [PACT 2011]

- If program has predefined parallel phases (i.e., *disciplined parallelism*), caches can **self-invalidate** potentially stale data between phases
 - Keep any data you read in most recent phase or data you own; invalidate the rest
- Directory just tracks the owner; eliminates transient states

	<pre>// P1 barrier(); ... = A[]; barrier(); B[] = C[]; barrier(); ...</pre>	<pre>// P2 barrier(); D[] = ...; barrier(); ... = A[]; barrier(); C[] = ...;</pre>	why is it ok to keep C in P1 cache? if P2 writes C next, P1 is guaranteed to not read C
P1 invalidates: A	-----	-----	P2 invalidates: -
	<pre>...</pre>		
	P1 will invalidate C here, since not read in this phase		

DeNovoSync

Can extend to DRF programs with arbitrary synchronization [ASPLOS 2015]

- Caches self-invalidate stale data at synch points (i.e., release/unblock)
- **Serialize reads** to memory locations used as synch variables
 - Reading synch variable requires registration at directory (*single-reader invariant*)
- Extends well to coherence for GPUs and heterogeneous systems

DeNovoSync

Can extend to DRF programs with arbitrary synchronization [ASPLOS 2015]

- Caches self-invalidate stale data at synch points (i.e., release/unblock)
- **Serialize reads** to memory locations used as synch variables
 - Reading synch variable requires registration at directory (*single-reader invariant*)
- Extends well to coherence for GPUs and heterogeneous systems

// P1

```
    ... = A[];  
    signal(*x);  
    B[] = C[];  
    wait(*y);  
    D[] =...;
```

// P2

```
    ...  
    wait(*x);  
    ___ = ___;  
    signal(*y);  
    ...
```

DeNovoSync

Can extend to DRF programs with arbitrary synchronization [ASPLOS 2015]

- Caches self-invalidate stale data at synch points (i.e., release/unblock)
- **Serialize reads** to memory locations used as synch variables
 - Reading synch variable requires registration at directory (*single-reader invariant*)
- Extends well to coherence for GPUs and heterogeneous systems

// P1

```
... = A[];  
signal(*x);  
B[] = C[];  
wait(*y);  
D[] = ...;
```

// P2

```
...  
wait(*x);  
___ = ___;  
signal(*y);  
...
```

what can P2 read here?

DeNovoSync

Can extend to DRF programs with arbitrary synchronization [ASPLOS 2015]

- Caches self-invalidate stale data at synch points (i.e., release/unblock)
- **Serialize reads** to memory locations used as synch variables
 - Reading synch variable requires registration at directory (*single-reader invariant*)
- Extends well to coherence for GPUs and heterogeneous systems

```
// P1                                // P2
... = A[];                            ...
signal(*x); → wait(*x);                what can P2 read here?
    B[] = C[];                        ___ = ___;
wait(*y); ← signal(*y);
    D[] = ...;                        ...
```

DeNovoSync

Can extend to DRF programs with arbitrary synchronization [ASPLOS 2015]

- Caches self-invalidate stale data at synch points (i.e., release/unblock)
- **Serialize reads** to memory locations used as synch variables
 - Reading synch variable requires registration at directory (*single-reader invariant*)
- Extends well to coherence for GPUs and heterogeneous systems

```
// P1                                // P2
... = A[]; before
signal(*x); → wait(*x);
B[] = C[];
wait(*y); ← signal(*y);
D[] = ...; after
```

what can P2 read here?

DeNovoSync

- Can extend to DRF programs with arbitrary synchronization [ASPLOS 2015]
- Caches self-invalidate stale data at synch points (i.e., release/unblock)
 - **Serialize reads** to memory locations used as synch variables
 - Reading synch variable requires registration at directory (*single-reader invariant*)
 - Extends well to coherence for GPUs and heterogeneous systems

```
// P1                                // P2
... = A[]; before
signal(*x); → wait(*x);
    B[] = C[];
wait(*y); ← signal(*y);
D[] = ...; after
```

what can P2 read here? A, C, D

DeNovoSync

Can extend to DRF programs with arbitrary synchronization [ASPLOS 2015]

- Caches self-invalidate stale data at synch points (i.e., release/unblock)
- **Serialize reads** to memory locations used as synch variables
 - Reading synch variable requires registration at directory (*single-reader invariant*)
- Extends well to coherence for GPUs and heterogeneous systems

// P1

```
... = A[];  
signal(*x);  
B[] = C[];  
wait(*y);  
D[] = ...;
```

// P2

```
...  
wait(*x);  
___ = ___;  
signal(*y);  
...
```

what can P2 write here?

DeNovoSync

Can extend to DRF programs with arbitrary synchronization [ASPLOS 2015]

- Caches self-invalidate stale data at synch points (i.e., release/unblock)
- **Serialize reads** to memory locations used as synch variables
 - Reading synch variable requires registration at directory (*single-reader invariant*)
- Extends well to coherence for GPUs and heterogeneous systems

```
// P1                                // P2
... = A[]; before
signal(*x); → wait(*x);
B[] = C[];
wait(*y); ← signal(*y);
D[] = ...; after
```

what can P2 write here?

DeNovoSync

- Can extend to DRF programs with arbitrary synchronization [ASPLOS 2015]
- Caches self-invalidate stale data at synch points (i.e., release/unblock)
 - **Serialize reads** to memory locations used as synch variables
 - Reading synch variable requires registration at directory (*single-reader invariant*)
 - Extends well to coherence for GPUs and heterogeneous systems

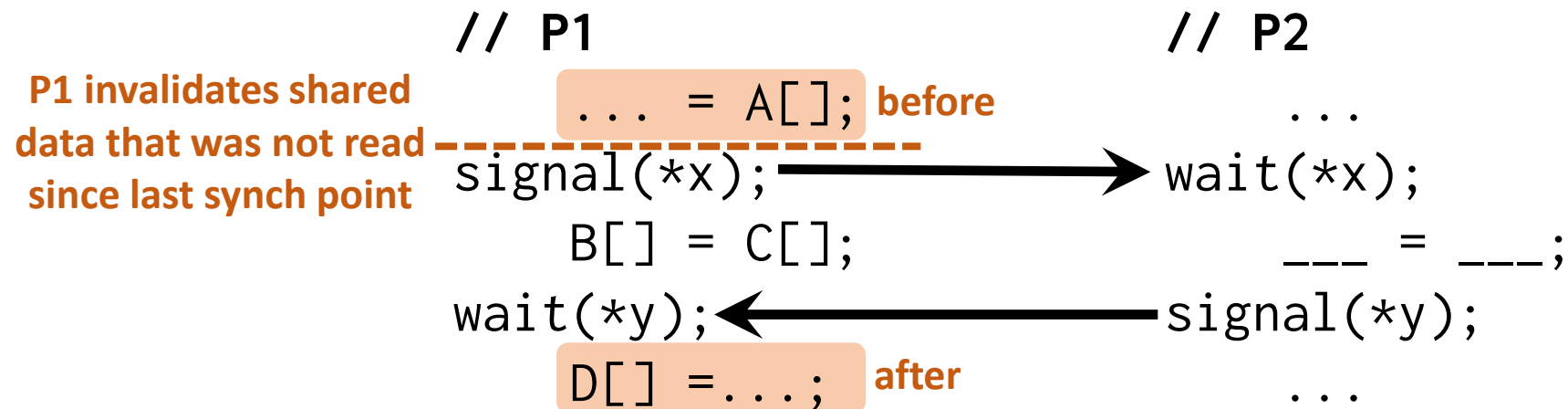
```
// P1                                // P2
... = A[]; before
signal(*x); → wait(*x);
    B[] = C[];          ___ = ___;
wait(*y); ← signal(*y);
D[] = ...; after
```

what can P2 write here? A, D

DeNovoSync

Can extend to DRF programs with arbitrary synchronization [ASPLOS 2015]

- Caches self-invalidate stale data at synch points (i.e., release/unblock)
- **Serialize reads** to memory locations used as synch variables
 - Reading synch variable requires registration at directory (*single-reader invariant*)
- Extends well to coherence for GPUs and heterogeneous systems



Relaxed Consistency

S. V. Adve and K. Gharachorloo, *Shared Memory Consistency Models: A Tutorial*, Western Research Laboratory Technical Report, 1995.

Relaxation	$W \rightarrow R$ Order	$W \rightarrow W$ Order	$R \rightarrow RW$ Order	Read Others' Write Early	Read Own Write Early
SC [16]					✓
IBM 370 [14]	✓				
TSO [20]	✓				✓
PC [13, 12]	✓			✓	✓
PSO [20]	✓	✓			✓
WO [5]	✓	✓	✓		✓
RCsc [13, 12]	✓	✓	✓		✓
RCpc [13, 12]	✓	✓	✓	✓	✓
Alpha [19]	✓	✓	✓		✓
RMO [21]	✓	✓	✓		✓
PowerPC [17, 4]	✓	✓	✓	✓	✓

Relaxed Consistency

Possible with SC: A=0, B=0, C=0, D=0, E=0, F=0?

```
// init
```

```
*x = 0; *y = 0;
```

```
// P1
```

```
*x = 1;
```

```
A = *y;
```

```
// P2
```

```
*y = 1;
```

```
B = *x;
```

```
// P3
```

```
C = *x;
```

```
D = *y;
```

```
// P4
```

```
E = *y;
```

```
F = *x;
```

i.

A	B	C	D	E	F	SC?
0	0	0	0	0	0	

Relaxed Consistency

Possible with SC: A=0, B=0, C=0, D=0, E=0, F=0?

```
// init
```

```
*x = 0; *y = 0;
```

```
// P1
```

```
*x = 1;  
A = *y;
```

```
// P2
```

```
*y = 1;  
B = *x;
```

```
// P3
```

```
C = *x;  
D = *y;
```

```
// P4
```

```
E = *y;  
F = *x;
```

i.

A	B	C	D	E	F	SC?
0	0	0	0	0	0	

Relaxed Consistency

Possible with SC: A=0, B=0, C=0, D=0, E=0, F=0? **No**

```
// init
```

```
*x = 0; *y = 0;
```

```
// P1
```

```
*x = 1;
```

```
A = *y;
```

```
// P2
```

```
*y = 1;
```

```
B = *x;
```

```
// P3
```

```
C = *x;
```

```
D = *y;
```

```
// P4
```

```
E = *y;
```

```
F = *x;
```

i.

A	B	C	D	E	F	SC?
0	0	0	0	0	0	no

Relaxed Consistency

Possible with TSO: A=0, B=0, C=0, D=0, E=0, F=0?

- Relaxed write-to-read, with store atomicity

// init

`*x = 0; *y = 0;`

// P1

`*x = 1;`

`A = *y;`

// P2

`*y = 1;`

`B = *x;`

// P3

`C = *x;`

`D = *y;`

// P4

`E = *y;`

`F = *x;`

i.

A	B	C	D	E	F	SC?	TSO?
0	0	0	0	0	0	no	

Relaxed Consistency

Possible with TSO: A=0, B=0, C=0, D=0, E=0, F=0?

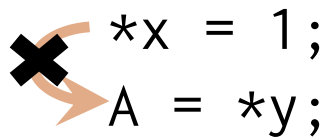
- Relaxed write-to-read, with store atomicity

// init

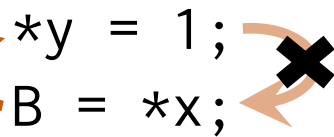
*x = 0; *y = 0;

relaxed write to read

// P1

 *x = 1;
A = *y;

// P2

 *y = 1;
B = *x;

// P3

C = *x;
D = *y;

// P4

E = *y;
F = *x;

i.

A	B	C	D	E	F	SC?	TSO?
0	0	0	0	0	0	no	

Relaxed Consistency

Possible with TSO: A=0, B=0, C=0, D=0, E=0, F=0? **Yes**

- Relaxed write-to-read, with store atomicity

```
// init
```

```
*x = 0; *y = 0;
```

relaxed write to read

```
// P1
```

✗ `*x = 1;` **WAR** `*y = 1;` **✗**
`A = *y;` `B = *x;`

```
// P2
```

```
// P3
```

happens first

```
// P4
```

```
C = *x;  
D = *y;
```

```
E = *y;  
F = *x;
```

i.

A	B	C	D	E	F	SC?	TSO?
0	0	0	0	0	0	no	yes



Relaxed Consistency

Possible with PC: A=0, B=0, C=0, D=0, E=0, F=0?

- Relaxed write-to-read, without store atomicity

```
// init
```

```
*x = 0; *y = 0;
```

```
// P1
```

```
*x = 1;
```

```
A = *y;
```

```
// P2
```

```
*y = 1;
```

```
B = *x;
```

```
// P3
```

```
C = *x;
```

```
D = *y;
```

```
// P4
```

```
E = *y;
```

```
F = *x;
```

i.

A	B	C	D	E	F	SC?	TSO?	PC?
0	0	0	0	0	0	no	yes	

Relaxed Consistency

Possible with PC: A=0, B=0, C=0, D=0, E=0, F=0? **Yes**

- Relaxed write-to-read, without store atomicity

// init

*x = 0; *y = 0;

relaxed write to read

// P1

// P1
*x = 1;
A = *y;

// P2

*y = 1;
B = *x;

// P3

happens first

// P4

C = *x;
D = *y;

E = *y;
F = *x;

i.

A	B	C	D	E	F	SC?	TSO?	PC?
0	0	0	0	0	0	no	yes	yes



Relaxed Consistency

Possible with SC: A=0, B=0, C=1, D=0, E=1, F=0?

```
// init
```

```
*x = 0; *y = 0;
```

```
// P1
```

```
*x = 1;
```

```
A = *y;
```

```
// P2
```

```
*y = 1;
```

```
B = *x;
```

```
// P3
```

```
C = *x;
```

```
D = *y;
```

```
// P4
```

```
E = *y;
```

```
F = *x;
```

	A	B	C	D	E	F	SC?	TSO?	PC?
i.	0	0	0	0	0	0	no	yes	yes
ii.	0	0	1	0	1	0			

Relaxed Consistency

Possible with SC: A=0, B=0, C=1, D=0, E=1, F=0? **No**

```
// init
```

```
*x = 0; *y = 0;
```

```
// P1
```

```
*x = 1;
A = *y;
```

```
// P2
```

```
*y = 1;
B = *x;
```

```
// P3
```

```
C = *x;
D = *y;
```

```
// P4
```

```
E = *y;
F = *x;
```

	A	B	C	D	E	F	SC?	TSO?	PC?
i.	0	0	0	0	0	0	no	yes	yes
ii.	0	0	1	0	1	0	no		



Relaxed Consistency

Possible with TSO: A=0, B=0, C=1, D=0, E=1, F=0?

➤ Relaxed write-to-read, with store atomicity

// init

*x = 0; *y = 0;

// P1

*x = 1;

A = *y;

// P2

*y = 1;

B = *x;

// P3

C = *x;

D = *y;

// P4

E = *y;

F = *x;

	A	B	C	D	E	F	SC?	TSO?	PC?
i.	0	0	0	0	0	0	no	yes	yes
ii.	0	0	1	0	1	0	no		

Relaxed Consistency

Possible with TSO: A=0, B=0, C=1, D=0, E=1, F=0?

- Relaxed write-to-read, with store atomicity

// init

*x = 0; *y = 0;

relaxed write to read

// P1


*x = 1;
A = *y;

// P2

*y = 1;
B = *x;

// P3

C = *x;
D = *y;

// P4

E = *y;
F = *x;

	A	B	C	D	E	F	SC?	TSO?	PC?
i.	0	0	0	0	0	0	no	yes	yes
ii.	0	0	1	0	1	0	no		

Relaxed Consistency

Possible with TSO: A=0, B=0, C=1, D=0, E=1, F=0?

- Relaxed write-to-read, with store atomicity

// init

*x = 0; *y = 0;

// P1

*x = 1;
A = *y;

// P2

*y = 1;
B = *x;

// P3

RAW C = *x;
WAR D = *y;

// P4

RAW E = *y;
WAR F = *x;

	A	B	C	D	E	F	SC?	TSO?	PC?
i.	0	0	0	0	0	0	no	yes	yes
ii.	0	0	1	0	1	0	no		

Relaxed Consistency

Possible with TSO: A=0, B=0, C=1, D=0, E=1, F=0? **No**

➤ Relaxed write-to-read, with store atomicity

```
// init
```

```
*x = 0; *y = 0;
```

```
// P1
```

```
*x = 1;
```

```
A = *y;
```

```
// P2
```

```
*y = 1;
```

```
B = *x;
```

```
// P3
```

```
C = *x;
```

```
D = *y;
```

```
// P4
```

```
E = *y;
```

```
F = *x;
```

	A	B	C	D	E	F	SC?	TSO?	PC?
i.	0	0	0	0	0	0	no	yes	yes
ii.	0	0	1	0	1	0	no	no	



Relaxed Consistency

Possible with PC: A=0, B=0, C=1, D=0, E=1, F=0?

- Relaxed write-to-read, without store atomicity

```
// init
```

```
*x = 0; *y = 0;
```

```
// P1
```

```
*x = 1;
```

```
A = *y;
```

```
// P2
```

```
*y = 1;
```

```
B = *x;
```

```
// P3
```

```
C = *x;
```

```
D = *y;
```

```
// P4
```

```
E = *y;
```

```
F = *x;
```

	A	B	C	D	E	F	SC?	TSO?	PC?
i.	0	0	0	0	0	0	no	yes	yes
ii.	0	0	1	0	1	0	no	no	

Relaxed Consistency

Possible with PC: A=0, B=0, C=1, D=0, E=1, F=0?

- Relaxed write-to-read, without store atomicity

// init

*x = 0; *y = 0;

relaxed write to read

// P1


*x = 1; A = *y; *y = 1; B = *x;

without store atomicity:
can disagree on order of
writes to x and y

// P3

C = *x;
D = *y;

// P4

E = *y;
F = *x;

	A	B	C	D	E	F	SC?	TSO?	PC?
i.	0	0	0	0	0	0	no	yes	yes
ii.	0	0	1	0	1	0	no	no	

Relaxed Consistency

Possible with PC: A=0, B=0, C=1, D=0, E=1, F=0?

- Relaxed write-to-read, without store atomicity

```
// init
```

```
*x = 0; *y = 0;
```

```
// P1
```

```
*x = 1;
```

```
A = *y;
```

```
// P2
```

```
*y = 1;
```

```
B = *x;
```

```
// P3
```

```
C = *x;
```

```
D = *y;
```

```
// P4
```

```
E = *y;
```

```
F = *x;
```

without store atomicity:

*x = 1;



*y = 1;



*x = 1;

	A	B	C	D	E	F	SC?	TSO?	PC?
i.	0	0	0	0	0	0	no	yes	yes
ii.	0	0	1	0	1	0	no	no	

Relaxed Consistency

Possible with PC: A=0, B=0, C=1, D=0, E=1, F=0?

- Relaxed write-to-read, without store atomicity

```
// init
```

```
*x = 0; *y = 0;
```

```
// P1
```

```
*x = 1;
```

```
A = *y;
```

```
// P2
```

```
*y = 1;
```

```
B = *x;
```

```
// P3
```

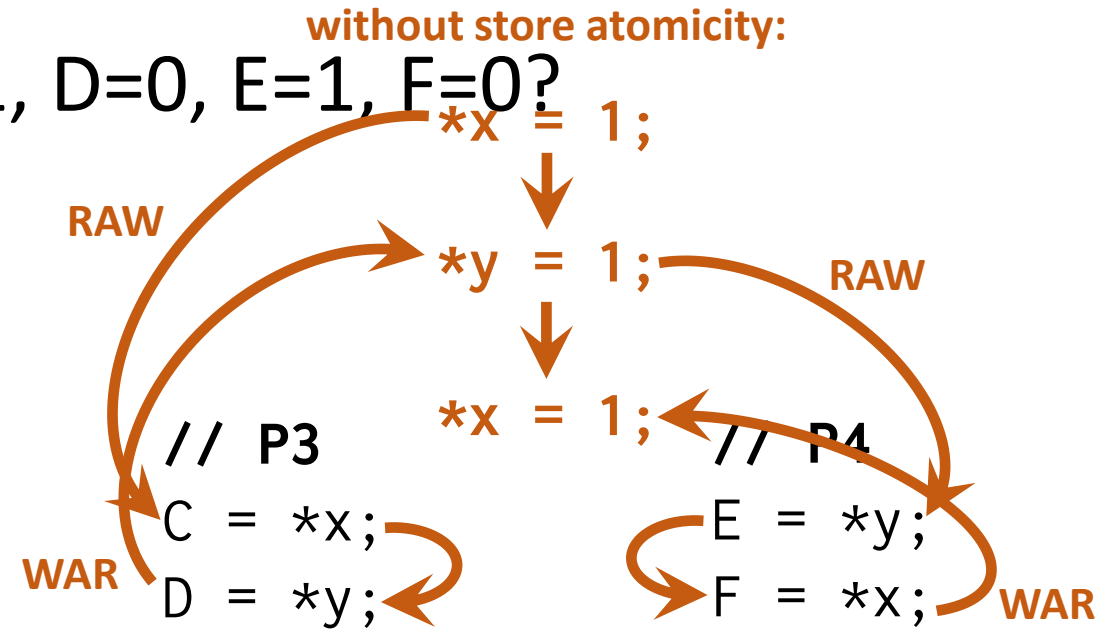
```
C = *x;
```

```
D = *y;
```

```
// P4
```

```
E = *y;
```

```
F = *x;
```



	A	B	C	D	E	F	SC?	TSO?	PC?
i.	0	0	0	0	0	0	no	yes	yes
ii.	0	0	1	0	1	0	no	no	

Relaxed Consistency

Possible with PC: A=0, B=0, C=1, D=0, E=1, F=0? **Yes**

➤ Relaxed write-to-read, without store atomicity

```
// init
```

```
*x = 0; *y = 0;
```

```
// P1
```

```
*x = 1;
```

```
A = *y;
```

```
// P2
```

```
*y = 1;
```

```
B = *x;
```

```
// P3
```

```
C = *x;
```

```
D = *y;
```

```
// P4
```

```
E = *y;
```

```
F = *x;
```

	A	B	C	D	E	F	SC?	TSO?	PC?
i.	0	0	0	0	0	0	no	yes	yes
ii.	0	0	1	0	1	0	no	no	yes



Relaxed Consistency

Possible with SC: A=0, B=1, C=0, D=1, E=0, F=1?

```
// init
```

```
*x = 0; *y = 0;
```

```
// P1
```

```
*x = 1;
```

```
A = *y;
```

```
// P2
```

```
*y = 1;
```

```
B = *x;
```

```
// P3
```

```
C = *x;
```

```
D = *y;
```

```
// P4
```

```
E = *y;
```

```
F = *x;
```

	A	B	C	D	E	F	SC?	TSO?	PC?
i.	0	0	0	0	0	0	no	yes	yes
ii.	0	0	1	0	1	0	no	no	yes
iii.	0	1	0	1	0	1			



Relaxed Consistency

Possible with SC: A=0, B=1, C=0, D=1, E=0, F=1? **Yes**

```
// init
```

```
*x = 0; *y = 0;
```

```
// P1
```

```
*x = 1;
```

```
A = *y;
```

```
// P2
```

```
*y = 1;
```

```
B = *x;
```

happens last

```
// P3
```

happens first

```
// P4
```

```
C = *x;
```

```
D = *y;
```

```
E = *y;
```

```
F = *x;
```

	A	B	C	D	E	F	SC?	TSO?	PC?
i.	0	0	0	0	0	0	no	yes	yes
ii.	0	0	1	0	1	0	no	no	yes
iii.	0	1	0	1	0	1	yes		



Relaxed Consistency

Possible with TSO/PC: A=0, B=1, C=0, D=1, E=0, F=1? **Yes**

```
// init
```

```
*x = 0; *y = 0;
```

```
// P1
```

```
*x = 1;
```

```
A = *y;
```

```
// P2
```

```
*y = 1;
```

```
B = *x;
```

```
// P3
```

```
C = *x;
```

```
D = *y;
```

```
// P4
```

```
E = *y;
```

```
F = *x;
```

	A	B	C	D	E	F	SC?	TSO?	PC?
i.	0	0	0	0	0	0	no	yes	yes
ii.	0	0	1	0	1	0	no	no	yes
iii.	0	1	0	1	0	1	yes	yes	yes

